

L^AT_EX News

Issue 24, February 2016 (L^AT_EX release 2016-02-01)

Contents

LuaT_EX support	1
Unicode data	1
More support for east European accents	2
Changes in Graphics	2
Changes in Tools	2
Improving support for Unicode engines	2

LuaT_EX support

This release refines the LuaT_EX support introduced in the 2015/10/01 release. A number of patches have been added to improve the behavior of `lATEX` (thanks largely to code review by Philipp Gesang). The kernel code has been adjusted to allow for changes in LuaT_EX v0.85–v0.88. Most notably, newer LuaT_EX releases allow more than 16 write streams and these are now enabled for use by `\newwrite`, but also the experimental `newtoken` Lua library has been renamed back to `token` which required small adjustments in the LuaT_EX setup.

The biggest change in LuaT_EX v0.85–v0.87 compared to previous versions is that all the primitives (originally defined in `pdfTEX`) dealing with the PDF “back end” are no longer defined, being replaced by a much smaller set of new primitives. This does not directly affect the core L^AT_EX files in this release but has required major changes to the `.ini` files used by T_EX Live and similar distributions to set up the format files. These changes in the LuaT_EX engine will affect any packages using these back end commands (packages such as `graphics`, `color`, `hyperref`, etc.). Until all contributed packages are updated to the new syntax users may need to add aliases for the old `pdfTEX` commands. A new `luapdftexalias` package has been contributed to CTAN (not part of the core L^AT_EX release) that may be used for this purpose.

See also the sections below for related changes in the tools and `graphics` bundles.

Unicode data

As noted in L^AT_EX News 22, the 2015/01/01 release of L^AT_EX introduced built-in support for extended T_EX systems. In particular, the kernel now loads appropriate

data from the Unicode Consortium to set `\lccode`, `\uccode`, `\catcode` and `\sfcode` values in an automated fashion for the entire Unicode range.

The initial approach taken by the team was to incorporate the existing model used by (plain) X_YT_EX and to pre-process the “raw” Unicode data into a ready-to-use form as `unicode-letters.def`. However, the relationship between the Unicode Consortium files and T_EX data structures is non-trivial and still being explored. As such, it is preferable to directly parse the original (`.txt`) files at point of use. The team has therefore “spun-out” both the data and the loading to a new generic package, `unicode-data`. This package makes the original Unicode Consortium data files available in the `texmf` tree (in `tex/generic/unicode-data`) and provides generic loaders suitable for reading this data into the plain, L^AT_EX 2_ε, and other, formats.

At present, the following data files are included in this new package:

- `CaseFolding.txt`
- `EastAsianWidth.txt`
- `LineBreak.txt`
- `MathClass.txt`
- `SpecialCasing.txt`
- `UnicodeData.txt`

These files are used either by L^AT_EX 2_ε or by `expl3` (i.e. they represent the set currently required by the team). The Unicode Consortium provides various other data files and we would be happy to add these to the generic package, as it is intended to provide a single place to collect this material in the `texmf` tree. Such requests can be mailed to the team as usual or logged at the package home page: <https://github.com/latex3/unicode-data>.

The new approach extends use of Unicode data in setting T_EX information in two ways. First, the `\sfcode` of all end-of-quotation/closing punctuation is now set to 0 (transparent to T_EX). Second, `\Umathcode` values are now set using `MathClass.txt` rather than setting up only letters (which was done using an arbitrary plane 0/plane 1 separation). There are also minor refinements to the existing code setting, particularly splitting the concepts of case and letter/non-letter category codes.

For X_YT_EX, users should note that `\xtxHanGlue` and `\xtxHanSpace` are *no longer defined*, that no

assignments are made to `\XeTeXinterchartoks` and that no `\XeTeXintercharclass` data is loaded into the format. The values which were previously inherited from the plain $\text{\XeTeX}$ setup files are *not* suitable for properly typesetting East Asian text. There are third-party packages addressing this area well, notably those in the CTeX bundle. Third-party packages may need adjustment to load the data themselves; see the `unicode-data` package for one possible loader.

More support for east European accents

As noted in $\text{\LaTeX}$ News 23, comma accent support was added for `s` and `t` in the 2015/10/01 release. In this release a matching `\textcommaabove` accent has been added for U+0123 (`\c{g}`, `g`) which is the lower case of U+0122 (`\c{G}`, `G`). In the OT1 and T1 encodings the combinations are declared as composites with the `\c` command, which matches the Unicode names “latin (capital|small) letter g with cedilla” and also allows `\MakeUppercase{\c{g}}` to produce `\c{G}`, as required. In T1 encoding, the composite of `\c` with `k`, `l`, `n` and `r` are also declared to use the comma below accent rather than cedilla to match the conventional use of these letters.

The UTF-8 `inputenc` option `utf8` has been extended to support all latin combinations that can be reasonably constructed with a (single) accent command and a base character for the T1 encoding so `g`, `u` and similar characters may be directly input using UTF-8 encoding.

Changes in Graphics

The changes in $\text{\LuaTeX}$ v0.87 mean that the `color` and `graphics` packages no longer share the `pdftex.def` file between $\text{\LuaTeX}$ and $\text{\pdfTeX}$. A separate file `luatex.def` (distributed separately) has been produced, and distributions are encouraged to modify `graphics.cfg` and `color.cfg` configuration files to default to the `luatex` option if $\text{\LuaTeX}$ v0.87 or later is being used. The team has contributed suitable `.cfg` files to CTAN to be used as models.

Normally it is best to let the local `graphics.cfg` automatically supply the right option depending on the $\text{\TeX}$ engine being used; however the `color` and `graphics` (and so `graphicx`) packages have been extended to have an explicit `luatex` option comparable to the existing `pdftex` and `xetex` options.

The `trig` package has been updated so that pre-computed values such as `\sin(90)` now expand to digits (1 rather than the internal token `\@one` in this case). This allows them to be used directly in PDF literal strings.

Changes in Tools

$\text{\LuaTeX}$ from version v0.87 no longer supports the

`\write18` syntax to access system commands. A new package `shellesc` has been added to `tools` that defines a new command `\ShellEscape` that may be used in all $\text{\TeX}$ variants to provide a consistent access to system commands. The package also defines `\write18` in $\text{\LuaTeX}$ so that it continues to access system commands as before; see the package documentation for details.

Improving support for Unicode engines

Stability concerns are always paramount when considering any change to the $\text{\LaTeX}$ 2 ϵ kernel. At the same time, it is important that the format remains usable and gives reliable results for users. For the Unicode $\text{\TeX}$ engines $\text{\XeTeX}$ and $\text{\LuaTeX}$ there are important differences in behavior from classical (8-bit) $\text{\TeX}$ engines which mean that identical default behaviors are not appropriate. Over the past 18 months the team has addressed the most pressing of these considerations (as detailed above and in $\text{\LaTeX}$ News 22 and 23), primarily by integrating existing patches into the kernel. There are, though, important areas which still need consideration, and which *may* result in refinements to kernel support in this area in future releases.

The default font setup in $\text{\LaTeX}$ 2 ϵ at present is to use the OT1 encoding. This assumes that hyphenation patterns have been read using appropriate codes: the T1 encoding is assumed. The commonly-used hyphenation patterns today, `hyph-utf8`, are set up in this way for 8-bit engines ($\text{\pdfTeX}$) but for Unicode engines use Unicode code points. This means that hyphenation will be incorrect with Unicode engines unless a Unicode font is loaded. This requires a concept of a Unicode font encoding, which is currently provided by the `fontspec` package in two versions, EU1 and EU2. The team is working to fully understand what is meant by a “Unicode font encoding”, as unlike a classical $\text{\TeX}$ encoding it is essentially impossible to know what glyphs will be provided (though each slot is always defined with the same meaning). There is also an overlap between this area and ideas of language and writing system, most obviously in documents featuring mixed scripts (for example Latin and Cyrillic).

As well as these font considerations, the team is also exploring to what extent it is possible to allow existing (8-bit) documents to compile directly with Unicode engines without requiring changes in the sources. Whether this is truly possible remains an open question.

It is important to stress that changes will only be made in this area where they do *not* affect documents processed with ϵ - $\text{\TeX}$ / $\text{\pdfTeX}$ (i.e. documents which are written for “classical” 8-bit $\text{\TeX}$ engines). Changes will also be made only where they clearly address deficiencies in the current setup for Unicode engines (i.e. where current behaviors are wrong).